

Proiectarea asamblorului aferent unui procesor CISC

Oleksik Vlad-Andrei
Calculatoare | C_31/1

1. Formatul instrucțiunii

Setul de instrucțiuni proiectat pentru procesorul implementat cuprinde instrucțiuni codificate pe 16 biți, fiecare încadrată într-una din 4 clase.

1.1. Clasa B1 (instrucțiuni cu 2 operanzi)

Clasa B1 respectă următorul format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	OPCODE			MAS		RS				MAD		RD			

Această clasă se identifică, prin urmare, prin bitul cel mai semnificativ ce are valoarea 0. Modurile de adresare pentru operanzii acestei clase de instrucțiuni sunt următoarele:

1.1.1. MAS pentru clasa B1

Operandul sursă poate fi adresat în următoarele moduri (se consideră PC adresa instrucțiunii curente):

Tabelul 1. Modurile de adresare sursă pentru clasa B1.

MAS	Adresare	Valoare operand sursă
00 ₍₂₎	AM (Adresare imediată)	Mem[PC+2]
01 ₍₂₎	AD (Adresare directă)	Reg[RS]
10 ₍₂₎	AI (Adresare indirectă)	Mem[Reg[RS]]
11 ₍₂₎	AX (Adresare indexată)	Mem[Reg[RS]+Mem[PC+2]]

Adresarea imediată

În cazul adresării imediate, operandul se găsește în memorie, pe cuvântul imediat următor instrucțiunii curente. Valoarea câmpului RS nu este definită.

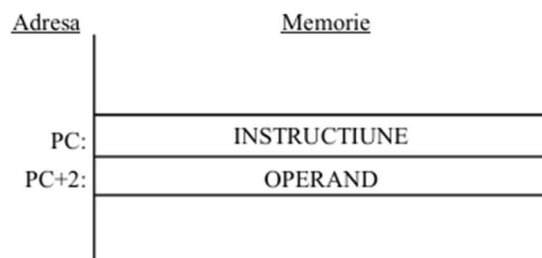


Fig. 1. Adresarea imediată

Adresare directă

În acest caz, valoarea operandului se găsește în registrul general al cărui număr este codificat pe cei 4 biți din câmpul RS.

Adresare indirectă

În acest caz, valoarea operandului se găsește în memorie, la adresa din registrul general al cărui număr este codificat pe cei 4 biți din câmpul RS.

Adresare indexată

În acest caz, valoarea operandului se găsește în memorie, la adresa dată de cuvântul imediat următor instrucțiunii curente, careia i se adaugă valoarea (indexul) din registrul general al cărui număr este codificat pe cei 4 biți din câmpul RS.

1.1.2. MAD pentru clasa B1

Operandul destinație poate fi adresat în următoarele moduri (se consideră PC adresa instrucțiunii curente):

Tabelul 2. Modurile de adresare sursă pentru clasa B1.

MAD	Adresare	Valoare operand dest.
00 ₍₂₎	Illegal	-
01 ₍₂₎	AD (Adresare directă)	Reg[RD]
10 ₍₂₎	AI (Adresare indirectă)	Mem[Reg[RD]]
11 ₍₂₎	AX (Adresare indexată)	Mem[Reg[RD]+Mem[PC+2*]]

Adresare directă

În acest caz, valoarea operandului se găsește în registrul general al cărui număr este codificat pe cei 4 biți din câmpul RD.

Adresare indirectă

În acest caz, valoarea operandului se găsește în memorie, la adresa din registrul general al cărui număr este codificat pe cei 4 biți din câmpul RD.

Adresare indexată

În acest caz, valoarea operandului se găsește în memorie, la adresa dată de cuvântul imediat următor instrucțiunii curente, căreia i se adaugă valoarea (indexul) din registrul general al cărui număr este codificat pe cei 4 biți din câmpul RD.

***Notă:** Dacă MAS este AM sau AX, la adresa PC+2 se va găsi indexul/valoarea imediată sursă, urmând ca, la adresa PC+4 să se regăsească indexul destinație (a se vedea Fig. 3.).

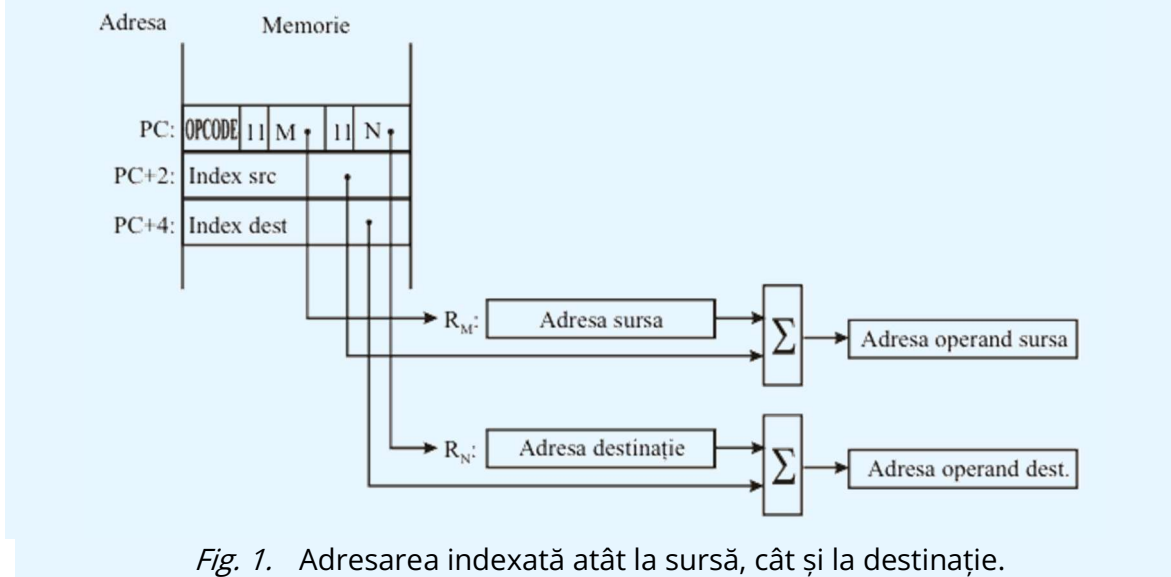


Fig. 1. Adresarea indexată atât la sursă, cât și la destinație.

1.1.3. Instrucțiunile din clasa B1

Opcode	Instrucțiune	Observații
000 ₍₂₎	MOV	op. dest. $\leftarrow$ op. src.
001 ₍₂₎	ADD	op. dest. $\leftarrow$ op. dest. + op. src.; FLAG $\leftarrow$ cond. codes
010 ₍₂₎	SUB	op. dest. $\leftarrow$ op. dest. - op. src.; FLAG $\leftarrow$ cond. codes
011 ₍₂₎	CMP	FLAG $\leftarrow$ cond. codes (op. dest - op. src.)
100 ₍₂₎	AND	op.dest. $\leftarrow$ op.dest. AND op.src.; FLAG $\leftarrow$ cond. codes
101 ₍₂₎	OR	op.dest. $\leftarrow$ op.dest. OR op.src.; FLAG $\leftarrow$ cond. codes
110 ₍₂₎	XOR	op.dest. $\leftarrow$ op.dest. XOR op.src.; FLAG $\leftarrow$ cond. codes

111₍₂₎ | Nefolosit

Notă: Instrucțiunea CMP permite adresarea destinație imediată.

1.2. Clasa B2 (instrucțiuni cu 1 operand)

Clasa B2 respectă următorul format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	OPCODE				0	0	MAD		RD			

Această clasă se identifică, prin urmare, prin biții cei mai semnificativi având valoarea 10₍₂₎. Modurile de adresare pentru operandul acestei clase de instrucțiuni sunt următoarele:

1.2.1. MAD pentru clasa B2

Operandul destinație poate fi adresat în următoarele moduri (se consideră PC adresa instrucțiunii curente):

Tabelul 3. Modurile de adresare sursă pentru clasa B1.

MAD	Adresare	Valoare operand dest.
00 ₍₂₎	Illegal	-
01 ₍₂₎	AD (Adresare directă)	Reg[RD]
10 ₍₂₎	AI (Adresare indirectă)	Mem[Reg[RD]]
11 ₍₂₎	AX (Adresare indexată)	Mem[Reg[RD]+Mem[PC+2]]

Adresare directă

În acest caz, valoarea operandului se găsește în registrul general al cărui număr este codificat pe cei 4 biți din câmpul RD.

Adresare indirectă

În acest caz, valoarea operandului se găsește în memorie, la adresa din registrul general al cărui număr este codificat pe cei 4 biți din câmpul RD.

Adresare indexată

În acest caz, valoarea operandului se găsește în memorie, la adresa dată de cuvântul imediat următor instrucțiunii curente, căreia i se adaugă valoarea (indexul) din registrul general al cărui număr este codificat pe cei 4 biți din câmpul RD.

1.2.2. Instrucțiunile din clasa B2

Opcode	Instrucțiune	Observații
0000 ₍₂₎	CLR	op. dest. $\leftarrow 0$
0001 ₍₂₎	NEG	op. dest. $\leftarrow$ (op. dest. XOR 0xFFFF)
0010 ₍₂₎	INC	op. dest. $\leftarrow$ op. dest. + 1
0011 ₍₂₎	DEC	op. dest. $\leftarrow$ op. dest. - 1
0100 ₍₂₎	ASL	Shiftare aritmetică la stânga
0101 ₍₂₎	ASR	Shiftare aritmetică la dreapta
0110 ₍₂₎	LSR	Shiftare logică la dreapta
0111 ₍₂₎	ROL	Rotire la stânga
1000 ₍₂₎	ROR	Rotire la dreapta
1001 ₍₂₎	RLC	Rotire prin <i>Carry</i> la stânga
1010 ₍₂₎	RRC	Rotire prin <i>Carry</i> la dreapta
1011 ₍₂₎	JMP	PC $\leftarrow$ op. dest.
1100 ₍₂₎	CALL	PUSH PC; PC $\leftarrow$ op. dest.;
1101 ₍₂₎	PUSH	SP $\leftarrow$ SP - 2; Mem[SP] $\leftarrow$ op. dest.
1110 ₍₂₎	POP	op. dest. $\leftarrow$ Mem[SP]; SP $\leftarrow$ SP + 2
1111 ₍₂₎	Nefolosit	

Notă: Instrucțiunile JMP, CALL, PUSH permit adresarea destinație imediată.

1.3. Clasa B3 (instrucțiuni de ramificație)

Clasa B3 respectă următorul format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	OPCODE				OFFSET							

Această clasă se identifică, prin urmare, prin biții cei mai semnificativi având valoarea 110₍₂₎. În condițiile în care acești biți codifică o instrucțiune de ramificație, câmpul OFFSET reprezintă o valoare pe 8 biți, cu semn (între -128 și 127, inclusiv). Dacă saltul este executat, se va sări la adresa PC+OFFSET.

Notă: Prin PC se înțelege adresa instrucțiunii care urmează instrucțiunii de ramificație.

1.3.1. Instrucțiunile din clasa B3

Opcode	Instrucțiune	Observații
0000 ₍₂₎	BR	Salt necondiționat.
0001 ₍₂₎	BNE	Salt dacă Z=0.
0010 ₍₂₎	BEQ	Salt dacă Z=1.
0011 ₍₂₎	BPL	Salt dacă S=0.
0100 ₍₂₎	BMI	Salt dacă S=1.
0101 ₍₂₎	BCS	Salt dacă C=1.
0110 ₍₂₎	BCC	Salt dacă C=0.
0111 ₍₂₎	BVS	Salt dacă V=1.
1000 ₍₂₎	BVC	Salt dacă V=0.
1001 ₍₂₎	Nefolosit	
1010 ₍₂₎		
1011 ₍₂₎		
1100 ₍₂₎		
1101 ₍₂₎		
1110 ₍₂₎		
1111 ₍₂₎		

1.4. Clasa B4 (instrucțiuni diverse)

Clasa B4 respectă următorul format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	OPCODE				TAG							

Această clasă se identifică, prin urmare, prin biții cei mai semnificativi având valoarea 111₍₂₎. Pentru fiecare instrucțiune, există o pereche de valori OPCODE-TAG care să o identifice. Instrucțiunile din clasa B4 se regăsesc în tabelul de mai jos.

1.4.1. Instrucțiunile din clasa B4

Opcode	Tag	Instrucțiune	Observații
0000 ₍₂₎	11110111 ₍₂₎	CLC	FLAG ← FLAG AND TAG
0000 ₍₂₎	11111110 ₍₂₎	CLV	
0000 ₍₂₎	11111011 ₍₂₎	CLZ	
0000 ₍₂₎	11111101 ₍₂₎	CLS	
0000 ₍₂₎	11110000 ₍₂₎	CCC	

0001 ₍₂₎	00001000 ₍₂₎	SEC	
0001 ₍₂₎	00000001 ₍₂₎	SEV	
0001 ₍₂₎	00000100 ₍₂₎	SEZ	FLAG ← FLAG OR TAG
0001 ₍₂₎	00000010 ₍₂₎	SES	
0001 ₍₂₎	00001111 ₍₂₎	SCC	
0010 ₍₂₎	Nedefinit	NOP	-
0011 ₍₂₎		HALT	A(0)BPO
0100 ₍₂₎		Rezervat	EI (Enable Interrupts)
0101 ₍₂₎		Rezervat	DI (Disable Interrupts)
0110 ₍₂₎		PUSH PC	SP ← SP – 2; Mem[SP] ← PC
0111 ₍₂₎		POP PC	PC ← Mem[SP]; SP ← SP + 2
1000 ₍₂₎		PUSH FLAG	SP ← SP – 2; Mem[SP] ← FLAG
1001 ₍₂₎		POP FLAG	FLAG ← Mem[SP]; SP ← SP + 2
1010 ₍₂₎		RET	POP PC
1011 ₍₂₎		WAIT	while(NOT INTR);
1100 ₍₂₎		RETI	POP PC; POP FLAG
1101 ₍₂₎			
1110 ₍₂₎		Nefolosit	
1111 ₍₂₎			

2. Sintaxa asamblor

Un program este o înșiruire de linii cu un format specificat. Exemple de programe bine formulate se pot regăsi în **Anexa 1**.

2.1. Formatul liniei

Fiecare linie având următorul format:

[label:][instrucțiune][#comentariu]

Notă: Notăția „[...]” marchează un element opțional.

Aici, „**label**” se referă la o etichetă ce se regăsește exact o dată la începutul unei instrucțiuni. Orice linie se poate finaliza cu un comentariu.

2.2. Formatul instrucțiunii în limbaj de asamblare

Pentru fiecare din cele 4 clase de instrucțiuni, există un format al instrucțiunii în limbaj de asamblare:

- Clasa B1: `INSTRB1 opd, []ops`
- Clasa B2: `INSTRB2 op`
- Clasa B3: `INSTRB3 label`
- Clasa B4: `INSTRB4`

„op”, „opd” și „ops” se referă la operanzi, așa cum sunt definiți în secțiunile 1.2.1., 1.1.2. și 1.1.1., respectiv.

Notă: În cadrul unei instrucțiuni din clasa B1, ordinea specificării operanzilor în limbajul de asamblare este **operand destinație, operand sursă**.

2.3. Formatul operanzilor

Fiecare dintre operanzi respectă următoarele convenții de reprezentare (în funcție de modul de adresare):

- AM: `{val_dec}` sau `0x{val_hex}` (ex.: `12` sau `0xC`)
- AD: `{R_index}` (ex.: `R13`)
- AI: `( {R_index} )` (ex.: `(R13)`)
- AX: `val( {R_index} )` sau `( {R_index} )val` (ex.: `13(R12)` sau `(R12)13`)

Note: 1) Valorile valide pentru registre sunt cuprinse în intervalul **R0-R15**.

2) Intervalul de validitate pentru valorile întregi imediate este [-32768; 32767] (sau [0; 65535] pentru reprezentarea fără semn).

3) Etichetele-țintă pentru instrucțiunile de ramificație se pot afla la maxim 128 adrese în urma/127 adrese în fața adresei memorate în PC.

Așadar, parsarea unui program în limbajul de asamblare descris anterior se realizează conform următoarei diagrame de stări:

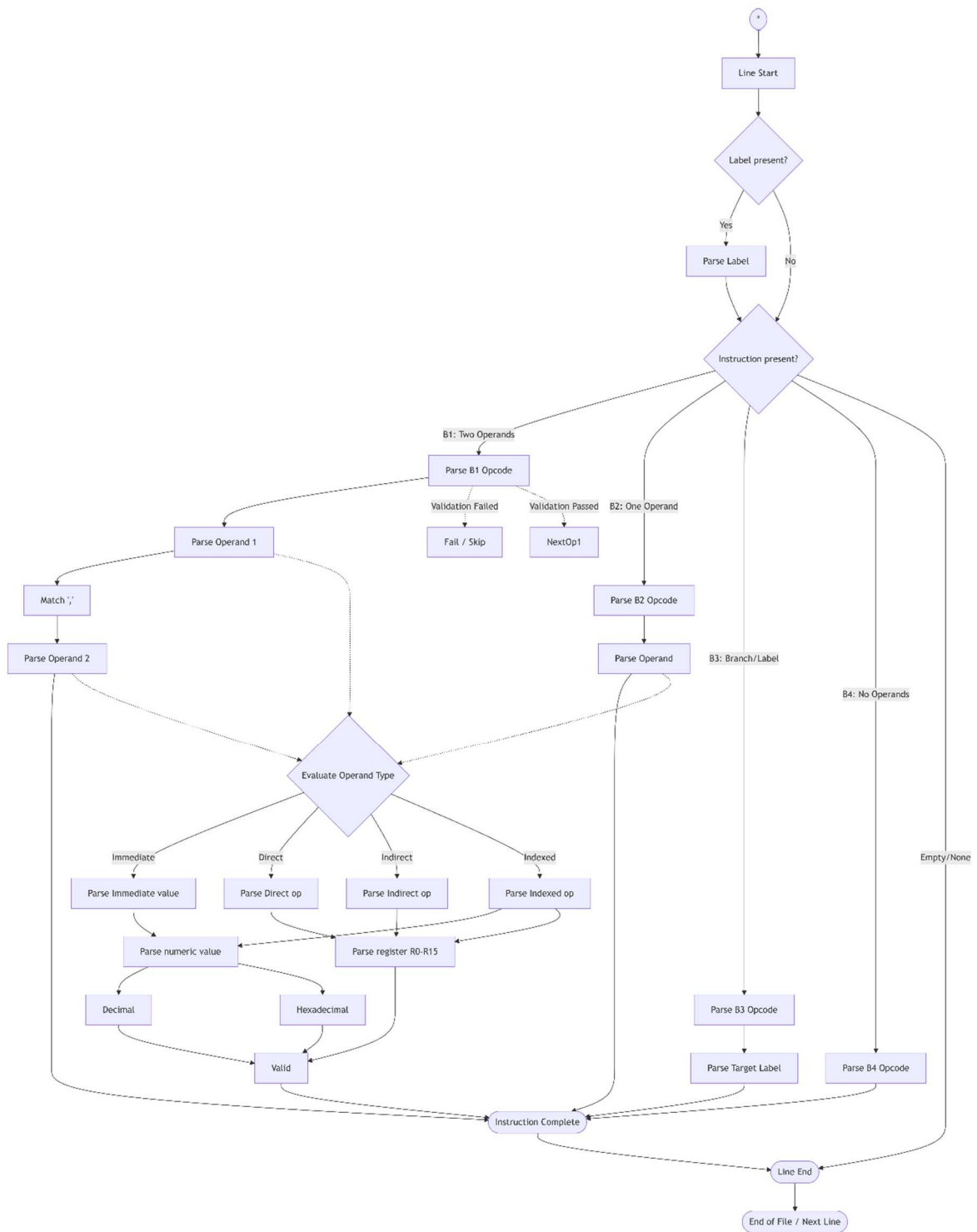


Fig. 2. Diagrama de stări a automatului asociat limbajului specificat

3. Mecanismul de translatare

Procesul de transformare a codului sursă din limbaj de asamblare în cod mașină este realizat printr-o arhitectură de asamblare în două etape (two-pass assembler). Această abordare este necesară pentru a permite utilizarea referințelor de tip "forward labeling", unde o instrucțiune de salt poate face referire la o etichetă care nu a fost încă definită în fluxul secvențial al programului.

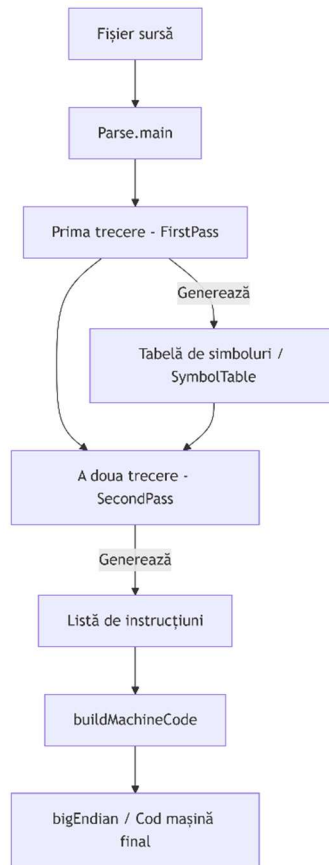


Fig. 3. Procesul de asamblare

3.1. Prima trecere

Prima etapă a algoritmului are rolul de a analiza structura liniară a programului și de a mapa topologia adreselor de memorie. Procesul începe cu inițializarea unui contor de locație la adresa zero. Pe măsură ce assemblerul parcurge fiecare linie furnizată de parser, acesta evaluează prezența etichetelor și a instrucțiunilor pentru a determina configurația memoriei.

Atunci când este detectată o etichetă la începutul unei linii, aceasta este introdusă într-o structură de date de tip dicționar, denumită tabela de simboluri. Adresa asociată etichetei corespunde valorii actuale a contorului de locație la momentul detectării. Dacă linia conține și o instrucțiune, asamblorul calculează dimensiunea acesteia și incrementează contorul de locație în consecință, pregătind adresa pentru următoarea linie.

Notă: Despre comportamentul sistemului în cazul etichetelor duplicate: Deoarece tabela de simboluri utilizează o structură de tip **map**, întâlnirea unei etichete deja existente va determina suprascrierea adresei anterioare cu cea nouă.

3.2. A doua trecere

Cea de-a doua etapă reprezintă faza de sinteză, în care se generează setul final de instrucțiuni executabile prin corelarea codului sursă cu informațiile extrase în prima etapă. În acest punct, asamblorul posedă cunoașterea completă a locațiilor de memorie pentru toate simbolurile din program.

Instrucțiunile care nu implică referințe simbolice sunt prelucrate direct. Provocarea principală în această etapă o reprezintă instrucțiunile de control al fluxului (cele din clasa B3), care utilizează adrese relative. Pentru aceste instrucțiuni, asamblorul extrage identificatorul etichetei și interoghează tabela de simboluri pentru a obține adresa de destinație. Valoarea finală inserată în instrucțiune este calculată ca un offset (deplasament) între adresa destinației și adresa curentă a instrucțiunii de salt.

Notă: În eventualitatea în care o instrucțiune face referire la o etichetă care nu a fost definită în prima etapă, algoritmul este configurat să utilizeze implicit o valoare nulă. Alternativa sigură este emiterea unei erori în această situație.

După finalizarea rezolvării tuturor referințelor, lista de instrucțiuni rezultată este supusă procesului de codificare binară. Aceasta presupune transformarea fiecărei instrucțiuni în cuvinte pe 16 biți și, în final, aplicarea formatului **Little Endian** pentru a asigura compatibilitatea cu arhitectura hardware țintă, inversând ordinea octeților pentru a plasa octetul cel mai semnificativ la adresa cea mai mică.

4. Implementarea asamblorului

Asamblorul este dezvoltat în limbajul Haskell, utilizând biblioteca Megaparsec pentru analiza sintactică, ceea ce asigură o gestionare robustă a erorilor de sintaxă. Pentru utilizarea utilitarului, este necesară prezența compilatorului GHC (varianta 9.4.x sau ulterioară) și a instrumentului Cabal. Procesul de instalare presupune compilarea surselor prin comanda `cabal build`, urmată de instalarea executabilului în sistem.

Interfața de linie de comandă permite un control granular asupra procesului de asamblare. Utilizatorul trebuie să specifice calea către fișierul sursă și locația dorită pentru fișierul binar prin parametrii `-i`, respectiv `-o`. Suplimentar, asamblorul oferă funcționalități de diagnosticare, precum vizualizarea tabelii de simboluri sau a reprezentării intermediare a instrucțiunilor parcurse, facilitând astfel procesul de debugging pentru dezvoltatorii de cod assembly.

Codul sursă al asamblorului implementat este disponibil atât anexat, cât și la adresa <https://github.com/vladoleksik/haskell-cisc-assembly>.

Codul sursă al unui emulator compatibil este disponibil la adresa <https://github.com/vladoleksik/cisc-emulator>, împreună cu un fișier de instalare pentru `windows_x86_64`.

Anexa 1 – Exemple de programe

Aceste programe au scopul de a testa îndeplinirea specificațiilor de către procesorul implementat.

Programul A

Acest program urmărește testarea sintaxei de asamblare și execuția instrucțiunilor de clase diferite. Se așteaptă compilarea, execuția și oprirea cu R2=1.

```
start: MOV (R6), 0xA #Tests assembler syntax
      SUB R6, 2
      MOV R2, 2(R6)
loop:  DEC R2
      CMP 1, R2
      NOP

redo:  BNE loop #loop condition
      HALT
```

Programul B

Acest program urmărește testarea setării codurilor de condiție și a execuției ramificațiilor și operațiilor pe stivă. Se așteaptă ca programul să se oprească cu R5=3, R1=-3.

```
MOV 2(R2), 3 #Tests branches and labels
SEZ
BEQ after
HALT
after:
PUSH 2(R2)
POP R5
NOP
SUB R1, 2(R2)
HALT
```

Programul C

Acest program urmărește testarea mecanismelor de apel și returnare din funcții. Se așteaptă ca programul să se oprească, calculând în R1 suma lui Gauss pentru N=100, respectiv 5050 (R1=0x13BA).

```

BR main # Tests calls and functions. Program should be loaded at 0x0.
# Expected: R1=0x13BA
gauss: MOV R1, R0
MOV R2, 0
loop: CMP R1, 0
BEQ end
ADD R2, R1
DEC R1
BR loop
end: MOV R0, R2
RET

main: MOV R3, 0x100
MOV (R3), 99
INC (R3)
MOV R0, (R3)
MOV R1, 2
CALL R1
MOV 2(R3), R0
MOV R1, R0
HALT

```

Programul D

Programul testează operațiile aritmetice (cu ALU).

```

MOV R0, 0x100 #Tests ALU ops, flags, branches. R0-R2 & R4-R5 should
# all be set.
INC 1(R0)
INC R0
CMP (R0), 0
BNE et1
HALT
et1: OR R1, 7
RRC R1
BCS et2
HALT
et2: MOV R2, R1
XOR R2, 7
NEG R2
BMI et3
HALT
et3: MOV R3, 2
CLR R3
CMP R3, 0
BPL et4
HALT
et4: MOV R4, 0x7FFF
AND R4, 0xFFFE

```

```

ADD R4, 5
BVS et5
HALT
et5: MOV R5, 5
ROR R5
HALT

```

Programul E

Programul testează instrucțiunile PUSH_FLAG, POP_FLAG, POP_PC. Se așteaptă ca R5 să aibă o valoare nenulă.

```

BR et0 #Tests PUSH_FLAG, POP_FLAG, PUSH_PC, flags, branches
MOV R2, 2
et0: CMP R0, 0
CLZ
BNE et1
HALT
et1: MOV R1, 1
MOV R3, 2
CMP R2, 2
BEQ et2
JMP R3
et2: MOV R4, 7
PUSH_PC
CMP R4, 7
PUSH_FLAG
CMP R4, 5
POP_FLAG
BNE et4
POP R5
et4: HALT

```

Programul F

Programul testează operații de shiftare și rotire. Se așteaptă ca R5=7, R0=6.

```

MOV R0, 3 #Tests shift and rotate ops, flags. Expected: halt with
# R5=7, R0=6
ASL R0
MOV R1, 0x8001
ASR R1
MOV R2, 0x8001
LSR R2
MOV R3, 0x8001
ROL R3
MOV R4, 0x8000
RLC R4

```

```

SEV
et1: BVC et2
MOV R0, 1
CLV
BR et1
et2: MOV R5, 7
HALT

```

Programul G

Programul testează instrucțiunea POP_PC. Se așteaptă ca programul să se finalizeze cu R2=7.

```

MOV R0, 0xA #Tests POP_PC. Meant to load from 0x0. Expected to halt
with R2=7.
PUSH R0
POP_PC
HALT
MOV R2, 7
HALT

```

Programul H

Programul testează instrucțiunea WAIT și mecanismul de tratare a întreruperilor. Se așteaptă ca programul să se termine cu R1=2 atunci când programul este încărcat începând de la 0x0.

```

BR main #This program busily waits until IRQ0 is activated, then halts
# with R1=2.
NOP #Expected to be loaded from 0x0
NOP
NOP
HALT #Here will be the address of int (0xA)!
int: POP R0
ADD R0, 2
PUSH R0
RETI
main: MOV R0, 0xA
MOV R1, 0x8 #Address of IV[IRQ0] (0x8)
MOV (R1), 0xA #Address of int handler for IRQ0 (0xA)
WAIT
MOV R1, 2
HALT

```